

MODULE 2

Conditionals and Recursion: Floor division and modulus, Boolean expressions, Logical operators, Conditional execution, Chained conditionals, Nested conditionals, Recursion, Stack Diagrams for Recursive Functions, Infinite recursion, Keyboard input; **Iterations:** Reassignment, Updating variables, The while statement, break. **Strings:** A string is a sequence, len, Traversal with a for loop, String slices, Strings are immutable, Searching, Looping and counting, String methods, String comparison

Self-Study Components: The in Operator

Textbook 1: Ch. 5, 7, 8

1. FLOOR DIVISION AND MODULUS

In Python, floor division and modulus are two important arithmetic operations used together, especially in number manipulation.

- i. **Floor Division (//):** Floor division divides and rounds down to nearest integer. It removes the decimal part and rounds down to the floor.

Syntax: `a // b`

Examples:

`print(10 // 3)` # Output: 3

- ii. **Modulus (%):** The modulus operator returns the remainder after division.

Syntax: `a % b`

Examples:

`print(10 % 3)` # Output: 1

PRACTICE PROGRAMS

1. Program to illustrate floor division and Modulus

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

floor_result = a // b
mod_result = a % b

print("Floor Division (a // b):", floor_result)
print("Modulus (a % b):", mod_result)
```

2. Program to check if a number is even or odd.

```
a = int(input("Enter a number: "))
if a%2 ==0:
    print("The number is Even ")
else:
    print("The number is Odd ")
```

3. Program to count digits in a number.

```
num = int(input("Enter a number: "))
count = 0

while num > 0:
    num = num // 10
    count = count + 1

print("Number of digits:", count)
```

2. BOOLEAN EXPRESSIONS

- A Boolean expression in Python is a statement that evaluates to one of two built-in values: True or False.
- These expressions are fundamental for controlling program flow using conditional statements (if, elif, else) and loops (while).
- Boolean expressions are created using comparison operators and logical operators.

Comparison Operators: These operators compare two values and return a Boolean result (True / False)

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 6</code>	True
<code>></code>	Greater than	<code>10 > 5</code>	True
<code><</code>	Less than	<code>10 < 9</code>	False
<code>>=</code>	Greater than or equal to	<code>2 >= 2</code>	True
<code><=</code>	Less than or equal to	<code>1 <= 2</code>	True

PRACTICE PROGRAMS

1. Program to illustrate the use of conditional operators in Boolean expressions.

```
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print(f'{x} == {y} :', x == y)
print(f'{x} != {y} :', x != y)
print(f'{x} > {y} :', x > y)
print(f'{x} < {y} :', x < y)
print(f'{x} >= {y} :', x >= y)
print(f'{x} <= {y} :', x <= y)
```

3. LOGICAL OPERATORS

They are used to combine or modify Boolean expressions.

i. and

Both conditions must be true. It returns False if any one of the inputs is False else returns True.

```
x = 5
print(x > 0 and x < 10) # True
```

ii. or

At least one condition true. It returns True if any one of the inputs is True else returns False.

```
n = 6
print(n % 2 == 0 or n % 3 == 0) # True
```

iii. not

Reverses condition. It only requires one argument and returns the negation of the argument i.e. returns the True for False and False for True.

```
x = 5
print(not(x > 10)) # True
```

PRACTICE PROGRAMS

1. Program to illustrate the use of Logical operators.

```
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print(f'{x} > 5 and {y} > 5 :', x > 5 and y > 5)
print(f'{x} > 5 or {y} > 5 :', x > 5 or y > 5)
print(f'not({x} > {y}) :', not(x > y))
```

4. CONDITIONAL EXECUTION

To write useful programs, we almost always need the ability to check conditions and change the behavior of the program accordingly. Conditional statements give us this ability. The simplest form is the if statement:

i. Simple if

If it is true, the indented statement runs. If not, nothing happens.

It executes a block of code if the given condition is true.

if statements have the same structure as function definitions: a header followed by an indented body.

Syntax:

```
if condition:
    statement 1
    statement 2
    .
    .
```

There is no limit on the number of statements that can appear in the body, but there has to be at least one.

Example:

```
x=10
if x>0:
    print(f'{x} is a positive number')
```

ii. if-else (Alternative Execution)

if else allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False.

Syntax:

```
if condition:
    statements
else:
    statements
```

Example:

```
x=int(input("Enter a number: "))
if x>0:
    print(f'{x} is a positive number')
else:
    print(f'{x} is a negative number')
```

iii. Nested if..else Conditional Statement

Nested if..else means an if-else statement inside another if statement.

We can use nested if statements to check conditions within conditions.

Syntax:

```
if condition:
    if condition:
        statements
    else:
        statements
else:
    statements
```

Example:

```
age= int (input("enter age"))
if age>=0:
    print("Positive number")
    if age!=0:
        print("The number is non – zero number")
    else:
        print("The number is a zero")
```

iv. Chained Conditionals - elif Statement

elif statement in Python stands for "else if."

It allows us to check multiple conditions, providing a way to execute different blocks of code based on which condition is true.

Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.

Syntax :

```
if condition:
    statements
elif condition:
    statements
elif condition:
    statements
else:
    statements
```

Example:

```
x=int(input("Enter a positive number less than 10: "))
if x>=10:
    print("Enter a number less than 10 !!!")
elif x>0:
    print(f'{x} is a positive number')
```

```

elif x<0:
    print(f'{x} is a negative number')
else:
    print("The number is Zero")

```

PRACTICE PROGRAMS	
1. Check is a number is an Armstrong number. <pre> num = int(input("Enter a number: ")) temp = num sum = 0 n = len(str(num)) # number of digits while temp > 0: digit = temp % 10 sum += digit ** n temp //= 10 if sum == num: print("Armstrong number") else: print("Not an Armstrong number") </pre>	
2. Largest of three numbers. <pre> a = int(input("Enter first number: ")) b = int(input("Enter second number: ")) c = int(input("Enter third number: ")) if a >= b and a >= c: print("Largest is:", a) elif b >= a and b >= c: print("Largest is:", b) else: print("Largest is:", c) </pre>	
3. Check if a number is a two-digit number. <pre> num = int(input("Enter a number: ")) if 10 <= abs(num) <= 99: print("Two-digit number") else: print("Not a two-digit number") </pre>	
4. Menu driven calculator to implement a simple calculator (+,-,*,/) <pre> print("1. Addition") print("2. Subtraction") print("3. Multiplication") print("4. Division") choice = int(input("Enter your choice: ")) a = float(input("Enter first number: ")) b = float(input("Enter second number: ")) if choice == 1: print("Result:", a + b) elif choice == 2: print("Result:", a - b) </pre>	

<pre> elif choice == 3: print("Result:", a * b) elif choice == 4: if b != 0: print("Result:", a / b) else: print("Division by zero not allowed") else: print("Invalid choice") </pre>
5. Basic Grading Program: Allot grade A,B,C,D,F based on the mark entered. <pre> marks = int(input("Enter marks: ")) if marks >= 90: print("Grade A") elif marks >= 75: print("Grade B") elif marks >= 60: print("Grade C") elif marks >= 50: print("Grade D") else: print("Grade F") </pre>
6. Check if an individual is eligible to vote and whether is a senior citizen <pre> age=int(input("Enter your age: ")) if age>=60: print("You are a senior citizen. Proceed to the priority queue for voting.") elif age >=18 print("You are eligible to vote. Please remain in the queue.") else: print("You are not eligible to vote") </pre>

5. RECURSION

A function calls itself to solve a problem by breaking it down into smaller, similar sub-problems.

Each recursive function must have two main components to prevent infinite loops:

Base Case: A condition that stops the recursion and provides a direct solution to the simplest instance of the problem.

Recursive Case: The part of the function that calls itself with a modified, smaller input, moving the solution progressively closer to the base case.

Syntax:

```

def recursive_function(parameters):
    if base_case_condition:
        return base_result
    else:
        return recursive_function(modified_parameters)

```

Example:

```

def factorial(n):

```

```

    if n == 0 or n == 1: # Base case: if n is 0 or 1, return 1
        return 1

    else: return n * factorial(n - 1) # Recursive case: n! = n * (n-1)!

num = 5

print(f"The factorial of {num} is {factorial(num)}") # Function call

```

Execution Flow for factorial(5):

1. factorial(5) calls factorial(4)
2. factorial(4) calls factorial(3)
3. factorial(3) calls factorial(2)
4. factorial(2) calls factorial(1)
5. factorial(1) hits the base case and returns 1
6. factorial(2) receives 1, calculates $2 * 1 = 2$, and returns 2
7. factorial(3) receives 2, calculates $3 * 2 = 6$, and returns 6
8. factorial(4) receives 6, calculates $4 * 6 = 24$, and returns 24
9. factorial(5) receives 24, calculates $5 * 24 = 120$, and returns 120

6. INFINITE RECURSION

Infinite recursion in Python occurs when a function repeatedly calls itself without a proper base case or termination condition.

This is a logical error, like an infinite loop, that consumes all available memory on the call stack, eventually causing the program to crash with a Recursion Error: “maximum recursion depth exceeded”

The primary causes of infinite recursion are:

- **Missing Base Case:** A function that calls itself must have a condition that eventually stops the recursion. Without it, the calls continue indefinitely.
- **Unreachable Base Case:** The logic within the function may fail to modify the parameters in a way that allows the base case condition to be met.
- For example, if a function is meant to stop when $n == 0$ but n is never decremented, the condition is never reached.

7. STACK DIAGRAM FOR RECURSION

```

def display(n, count):
    if count==0:
        return
    print("Your name is : ", n)
    count=count-1
    display(n,count)

```

```
n = input("Enter Your Name: ")
```

```
c= 3
```

```
display(n,c)
```

```
|-----|
```

```
| display frame      |
```

```
| n = "Anna"        |
```

```
| if count ==0       |
```

```
| base condition met |
```

```
| return            |
```

```
|-----|
```

```
| display frame      |
```

```
| n = "Anna"        |
```

```
| if count ==0       |
```

```
| count=1-1          |
```

```
| display (n,0)      |
```

```
|-----|
```

```
| display frame      |
```

```
| n = "Anna"        |
```

```
| if count ==        |
```

```
| count=2-1          |
```

```
| display (n,1)      |
```

```
|-----|
```

```
| display frame      |
```

```
| n = "Anna"        |
```

```
| if count ==0       |
```

```
| count=3-1          |
```

```
| display (n,2)      |
```

```
|-----|
```

```
| __main__ frame     |
```

```
| n = "Anna"        |
```

```
| c = 3              |
```

```
|display (n,c)       |
```

```
|-----|
```

Explanation

- ✓ The main frame calls display(n,3).
- ✓ Each function call creates a new stack frame.

- ✓ count decreases: $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$.
- ✓ When count == 0, the base condition is reached and the function returns.
- ✓ The stack unwinds back to __main__.

PRACTICE PROGRAMS

1. Program to print using recursion. (Infinite Recursion: No base condition)

```
def display(n):
    print("Your name is : ", n)
    display(n)

n = input("Enter Your Name: ")
display(n)
```

2. Program to print name N times using recursion.

```
def display(n, count):
    if count==0:          #base condition
        return
    print("Your name is : ", n)
    count=count-1
    display(n,count)

n = input("Enter Your Name: ")
c= int(input("Enter number of repetitions: "))
display(n,c)
```

3. Program to compute factorial of a number.

```
def factorial(n):
    if n == 0 or n == 1: # base case
        return 1
    else:
        return n * factorial(n-1)

num = int(input("Enter a number: "))
print("Factorial:", factorial(num))
```

4. Program to compute fibonacci.

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n = int(input("Enter position: "))

print("Fibonacci number:", fibonacci(n))
```

5. Program to print count down timer using recursion.

```
def countdown(n):
    print(".\n.\n.\n.\n")
    if n == 0:
        print("Time is Up")
        return
    print(n)
    countdown(n-1)
countdown(5)
```

6. Program to compute sum of first N numbers.

```
def sum(n):
```

```

        if n == 1:    # base case
            return 1
        else:
            return n + sum(n-1)

n = int(input("Enter a number: "))
print("Sum =", sum(n))

```

7. Program to compute power using recursion.

```

def power(a, b):
    if b == 0:
        return 1
    else:
        return a * power(a, b-1)

x = int(input("Enter base: "))
y = int(input("Enter exponent: "))
print("Result:", power(x, y))

```

8. KEYBOARD INPUTS

Python provides a built-in function called input that stops the program and waits for the user to type something.

When the user presses Return or Enter, the program resumes and input returns what the user typed as a string.

```
>>> text = input()
```

Input() can take a prompt as an argument:

```
>>> name = input("What...is your name?")
```

input() function by default returns a string.

If you expect the user to type an integer, you can try to convert the return value using type casting.

```

>>> num = int(input("Enter a number "))
>>> num = float(input("Enter a number "))

```

9. REASSIGNMENT AND UPDATING VARIABLE

Reassignment for a Variable in Python Reassignment means giving a new value to a variable that already has a value.

Reassignment overwrites the previous value.

No need to declare variables again.

Python is dynamically type

Updating variables in Python means changing the value of a variable during program execution.

Example:

```
x = 10
```

```
x = 20
```

```
print(x) # Output: 20
```

Reassignment with Expressions:

```
x = 5
```

```
x = x + 3
```

```
print(x) # Output: 8
```

Multiple Reassignments: Keeps the latest value.

```
name = "Alice"
```

```
name = "Bob"
```

```
name = "Charlie"
```

```
print(name) # Output: Charlie
```

Changing Data Types:

```
x = 10
```

```
x = "Hello"
```

```
x = 3.14
```

Shortcut Reassignment:

```
x += 5          # same as x = x + 5
```

Updating Multiple Variables:

```
a, b = 5, 10
```

```
a, b = b, a
```

```
print(a, b) # Output: 10 5 6.
```

10. WHILE LOOP – ITERATIONS

The while statement in Python is used to repeatedly execute a block of code if a condition is true.

Syntax:

```
while condition:
```

```
    statement1
```

```
    statement 2
```

The loop continues until the condition becomes False.

Executes while condition is True

Useful when number of iterations is not fixed

Must update variables to avoid infinite loops.

Example: While loop

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i += 1
```

Output: 1 2 3 4 5

Example 2: Infinite Loop

```
i = 1
```

```
while i <= 5:
```

```
    print(i)          # creates an infinite loop because i is never updated.
```

Example 3: Using break

```
i = 1
```

```
while i <= 10:
```

```
    if i == 5:
```

```
        break          # stops the loop once the condition is met.
```

```
print(i)
i += 1
```

Output: 1 2 3 4.

Example 4: Using continue

```
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue    # skips the current iteration.
    print(i)
```

Output: 1 2 4 5

11. BREAK STATEMENT

The break statement in Python is used to immediately terminate a loop (either for or while) when a certain condition is met. Stops the loop instantly. Control moves to the next statement after the loop.

Example: Using break in a while loop

```
i = 1
while i <= 10:
    if i == 5:
        break
    print(i)
    i += 1
```

12. STRING

A string is a sequence of characters enclosed in quotes (' ' or " "), which means it is an ordered collection of values.

You can access these characters one at a time using the bracket [] operator.

Python uses 0-based indexing.

```
S1= "Hello"
print(S1)      #Hello
print(S1[0])   #H
print(S1[1])   #e
print(S1[2])   #l
```

_____ **negative indices** _____

```
S1= "Hello"
print(S1[-1]) # counts backward from the end of the string
print(S1[-2])
print(S1[-3])
print(S1[-4])
```

```
print(S1[-5])
```

13. LENGTH OF STRING - len ()

The built-in len function returns the number of characters in a string.

```
>>> length = len(fruit)
```

```
>>> last = fruit[length]
```

Index Error: string index out of range To access the last character, you must subtract 1 from the length (fruitlength-1).

```
>>> last = fruit[length-1]
```

14. STRING TRAVERSAL

Traversal: Accessing the elements of a data structure one by one in a systematic manner is called traversal.

i. The while Loop Approach

This requires manual management of the index variable.

Example:

```
index = 0
```

```
fruit = "Apple"
```

```
while index < len(fruit):
```

```
    print(fruit[index])
```

```
    index = index + 1
```

ii. The for-Loop Approach

This is the more concise way to traverse a string in Python.

```
fruit = "Apple"
```

```
for letter in fruit:
```

```
    print(letter)
```

PRACTICE PROGRAMS

1. Generate the output using the given inputs.

Inputs

```
prefixes = "JKLM"
```

```
suffix = "ack"
```

Output

```
Jack
```

```
Kack
```

```
Lack
```

```
Mack
```

```
prefixes = "JKLM"
```

```
suffix = "ack"
```

```
for ch in prefixes:
```

```
    print(ch + suffix)
```

2. Print vowels in the given string.

```
string="HelloVVCE"
```

```
for i in string:
```

```
    if i in "aeiouAEIOU":
```

```
        print(i)
```

3. Print characters at even index.

<pre> string = "HelloVVCE" for i in range(len(string)): if i % 2 == 0: print(string[i]) </pre>
4. Count occurrence of a character in the string. <pre> string = input("Enter a string: ") char = input("Enter a character: ") count = 0 for ch in string: if ch == char: count = count+1 print("Occurrence:", count) </pre>
5. Count occurrence of each vowel in the given string. <pre> s = "Python Programming" ls= s.lower() print("Frequency of each vowel:") for ch in "aeiou": count = 0 for i in ls: if i == ch: count += 1 print(f'{ch}:{count}') </pre>
6. Program to count consonants and vowels in the given string. <pre> s = "Welcome" l= len(s) vowel=0 conso=0 print("The length of the string is :", l) for ch in s: if ch in "aeiouAEIOU": vowel=vowel+1 else: conso=conso+1 print("The number of Consonants:", conso) print("The number of Vowels:",vowel) </pre>

15. STRING SLICING

A segment of a string is called a slice.

You select a slice using the [n:m] ie. [start: end] syntax which returns the part of the string from the "n-th" character to the "m-th" character.

The Inclusive/Exclusive Rule: The slice includes the first index but excludes the last.

Example:

```

string="HelloVVCE"
print(string[1:4]) # ell
print(string[:4])  # Hell
print(string[4:])  # oVVCE
print(string[:])   # HelloVVCE
print(string[5:2]) # Prints an empty string "" - start must be less than end index

```

16. STRINGS ARE IMMUTABLE

Strings in Python are immutable, meaning you cannot change an existing string or modify a specific character index.

```
>>> greeting = 'Hello, world!'
```

```
>>> greeting[0] = 'J'
```

TypeError: 'str' object does not support item assignment.

To modify a string, you must create a new string object that is a variation of the original.

```
>>> greeting = 'Hello, world!'
```

```
>>> new_greeting = 'J' + greeting[1:]
```

```
>>> print(new_greeting)
```

Output: 'Jello, world!'

In this example, the original greeting remains unchanged in memory. The variable `new_greeting` points to an entirely different string object.

17. SEARCHING

Searching means checking whether a character or substring exists inside a string.

```
word = "hello"
```

```
letter = "h"
```

```
index = 0
```

```
found = -1
```

```
while index < len(word):
```

```
    if word[index] == letter:
```

```
        found = index
```

```
        break
```

```
    index = index + 1
```

```
if found != -1:
```

```
    print("The element Found at index: ", found )
```

```
else:
```

```
    print("Element not found")
```

18. STRING METHODS

Strings provide methods that perform a variety of useful operations.

A method is like a function—it takes arguments and returns a value—but the syntax is different.

i. **upper()**

Converts a string to upper case.

Example:

```
s="Python"
```

```
uc=s.upper()
```

```
print("Uppercase: ", uc)
```

ii. **lower()**

Converts a string to lower case.

Example:

```
s="Python"
```

```
lc= s.lower()
```

```
print("Lowercase: ", lc)
```

iii. **find()**

The find () method is used to search for a substring (or character) in a string and returns the index of its first occurrence.

Syntax:

find(substring, start, end)

start and end parameters are optional.

find(substring) – Valid

find (substring, start) – Valid

find (substring, end) - Invalid we cannot use end without start.

Parameters:

- search_string → The character or substring you want to find
- start (optional) → Starting index (default = 0)
- end (optional) → Ending index (default = length of string)

Return Value:

- Returns the index (position) of the first occurrence
- Returns -1 if the substring is not found

Example:

```
s="Python"
```

```
index= s.find("P")
```

```
if index != -1:
```

```
    print("The element found as index:", index)
```

```
else:
```

```
    print("Element Not Found")
```

PRACTICE PROGRAMS

1. Convert one half of string to upper and other to lower case.

```
s = input("Enter a string: ")
```

```
mid = len(s) // 2
```

```
result = s[0:mid].upper() + s[mid:len(s)].lower()
```

```
print("Converted string:", result)
```

2. Generate the given output from the given input string.

input: abcde

output: AbCdE

```
s = input("Enter a string: ")
```

```
result = ""
```

```
for i in range(len(s)):
```

```
    if i % 2 == 0:
```

```
        result += s[i].upper()
```

```
    else:
```



```
result += s[i]
```

```
print("Result:", result)
```

3. Program to find all occurrence of a character using find().

```
s = "banana"
```

```
ch = "a"
```

```
index = s.find(ch)
```

```
if index == -1:
```

```
    print("Element not Found")
```

```
while index != -1:
```

```
    print("The character occurs at index:", index)
```

```
    index = s.find(ch, index + 1)
```

19. STRING COMPARISON

The relational operators work on strings.

S1==S2 → checks if strings S1 and S2 are equal.

S1>S2 → checks if strings S1 is greater than S2.

S1<S2 → checks if strings S1 is less than S2.

Comparison is done character by character, based on ASCII values. If first characters are equal, moves to next characters.

Uppercase letters have smaller ASCII values than lowercase

So Lowercase letters are considered greater. "a" > "A"

```
S1 = "Apple"
```

```
S2 = "apple"
```

```
print(S1 < S2) # True (because 'A' < 'a')
```

Example:

```
word1 = input("Enter first word: ")
```

```
word2 = input("Enter second word: ")
```

```
# Convert both to lowercase for fair comparison
```

```
w1 = word1.lower()
```

```
w2 = word2.lower()
```

```
if w1 == w2:
```

```
    print("Both words are equal.")
```

```
elif w1 < w2:
```

```
    print(word1, "comes before", word2)
```

```
else:
```

```
    print(word1, "comes after", word2)
```